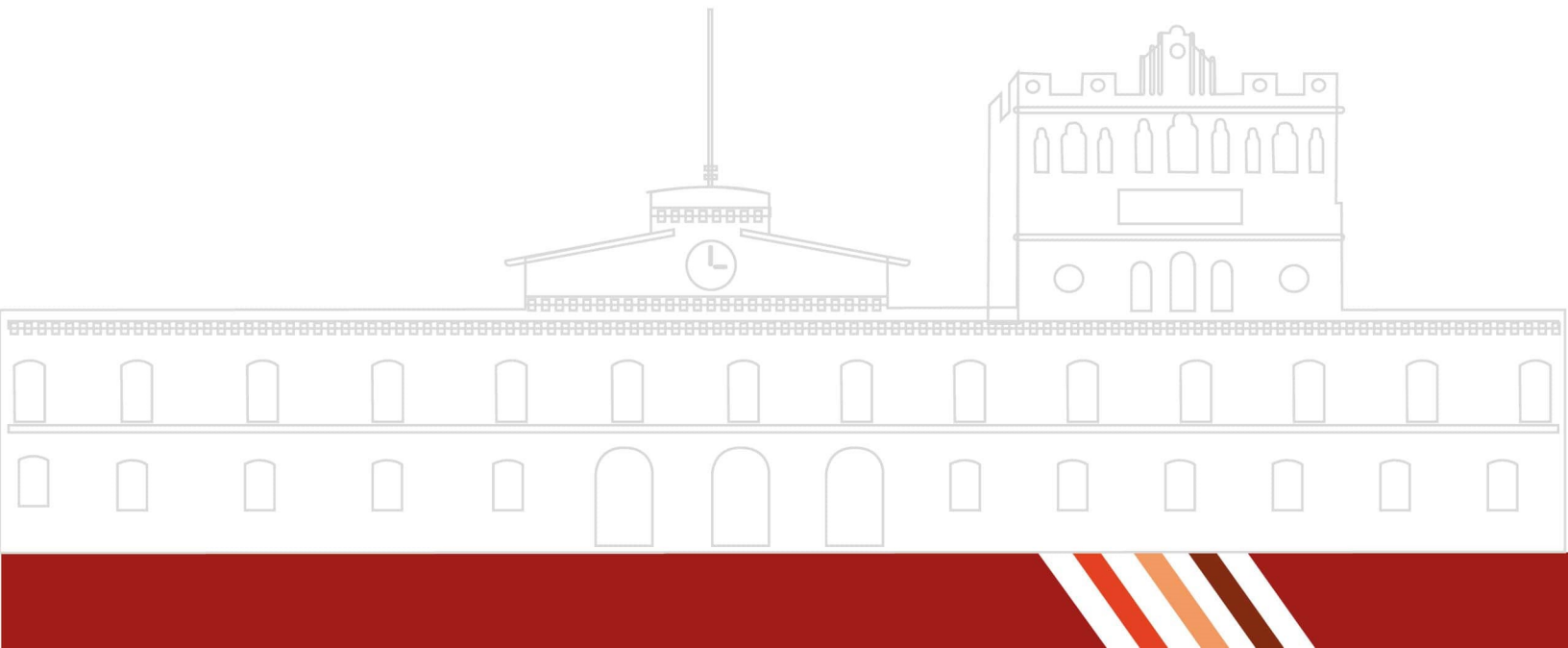


REPORTE DE PRÁCTICA NO. 3

Construcción de un Autómata Finito Determinista

ALUMNO:

Andrés García Colmenares



1. Introducción

Los Automatas Finitos son maquinas teoricas que cambian de un estado a otro, dependiendo de la entrada recibida de esta. Su salida es limitada, donde dependiendo de la entrada que se le ha dado, decide si es **Aceptada**, o **No Aceptada**

Para esta practica, se creara un Autómata Finito Determinista a traves de codigo, donde se definiran cada una de sus características, como sus elementos, funciones de transición y estados finales/iniciales

2. Marco teórico

Análisis de requerimientos

Para la creacion de un Automata Finito Determinista en codigo, se usara Python, y la libreria "automata-lib"

Python

Python es un lenguaje de programación versatil que es altamente usado, ya sea desarrollo web, analisis de datos o inteligencia artificial, al tener paquetes por medio de pip, que es el gestor de paquetes integrado en python [5]

Automata

Automata es una libreria para python 3, la cual implementa estructuras y algoritmos para la manipulacion de automatas, ademas de esto, está optimizado para entradas largas, y tiene soporte para vista grafica de automatas [4]

3. Herramientas empleadas

En esta practica, se han utilizado diferentes herramientas para la creacion de automatas finitos

1. Python. Lenguaje de programación con una gran biblioteca de librerias para la agilidad en programas y diferentes usos [5]
2. Automata. Libreria para la implementacion de Automatas Finitos Deterministas [4]

4. Desarrollo

Análisis de requisitos

Para la creación del automata finito, primero identificamos los componentes
Donde tenemos los siguientes valores

Alfabeto de Entrada: $\Sigma = (0, 1)$

Conjunto de Estados: $Q = (Q_1, Q_2, Q_3)$

Estado inicial: Q_1

Estado final: Q_2

Table 1: Tabla de Funciones de Transición

estados	0	1
q1	q1	q2
q2	q3	q2
q3	q2	q2

Traducción a código

Para la construcción del automata finito, usamos el siguiente código:

```
1 # Importacion de libreria
2 from automata.fa.dfa import DFA
3
4 # Creacion del automata
5 automata = DFA(
6     # Alfabeto de entrada
7     input_symbols={'0', '1'},
8     # Conjunto de estados
9     states={'q1', 'q2', 'q3'},
10    # Estados de transicion
11    transitions={
12        'q1': {'0': 'q1', '1': 'q2'},
13        'q2': {'0': 'q3', '1': 'q2'},
14        'q3': {'0': 'q2', '1': 'q2'}
15    },
16    # Estado inicial
17    initial_state='q1',
18    # Estado final
19    final_states={'q2'}
20 )
```

Con este código, nuestro automata tiene lo necesario para ejecutarse, pero aun falta algo, la función para darle cadenas y regresar una instrucción a partir de esto

```
1 # Funcion para entrada de caracteres en el automata finito determinista
2 def leerentrada(AFD):
3     try:
4         while True:
5             # Entrada de caracteres, Aceptado si es valido, Rechazado si falla
6             if AFD.accepts_input(input("Entrada:")):
7                 print("Aceptado")
8             else:
9                 print("Rechazado")
10    except KeyboardInterrupt:
11        print("")
12
13 leerentrada(automata)
```

Ahora, al terminar el código, podemos ejecutarlo en nuestra terminal y esperar lo siguiente

```
1 PS> python ./archivo.py
2 Entrada: 101
3 Aceptado
4 Entrada: 0
5 Rechazado
6 Entrada: 11
7 Aceptado
8 Entrada: 1000100001
9 Aceptado
10 ...
```

El código presente, se encuentra en mi propio github, que se accede desde este link

5. Cuestionario

¿Qué paradigma de programación utilizaste para la implementación del AFD?,

¿Cuáles elementos del lenguaje de programación utilizaste?

La programación imperativa, y en el lenguaje de programación, se utilizaron los elementos de funciones, paquetes, y variables

¿Qué estructuras de datos utilizaste para implementar los componentes del AFD?

Clases formadas por un paquete externo (automata-lib)

¿Qué elementos del lenguaje de programación te permitieron implementar las funciones de transición?, ¿cuáles son las limitaciones del programa que construiste?

Los paquetes y las clases, así como las funciones; una de las limitaciones es que el programa necesita cambiar de código para cambiar el AFD que se usa, así como que puede fallar debido a caracteres especiales en la terminal

¿Qué condiciones se deben cumplir para que el programa pueda evaluar cualquier AFD?

Editar el código fuente para cambiar los elementos del AFD completo, ya que no permite la edición en el tiempo de ejecución

6. Conclusiones

La actividad fue interesante, y ayudo a entender los paquetes en el lenguaje de programacion de python, asi como la programacion de estos en el ambito de nuestra carrera, a traves de esto se puede agregar mas informacion a un automata y darle una mayor profundidad si es necesario con nuevos casos o diferentes alfabetos

Referencias Bibliográficas

References

- [1] Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). Lenguajes formales y teoría de autómatas. *Editorial Alfaomega. Argentina.*
- [2] Ruiz Catalán, J. (2010). Compiladores: *teoría e implementación*. Editorial Alfaomega. Mexico.
- [3] Brookshear, J. G. (1995). Teoría de la Computación. *Lenguajes formales, autómatas y complejidad*. Editorial Addison-Wesley Iberoamericana. Primera edición. U.S.A.
- [4] Caleb Evans (2016-2025). Automata <https://caleb531.github.io/automata/>
- [5] Python. (2026). Python <https://www.python.org/>